デザインとプログラミング 2026
p5.jsで画像データを扱う - 画像の分析・再合成

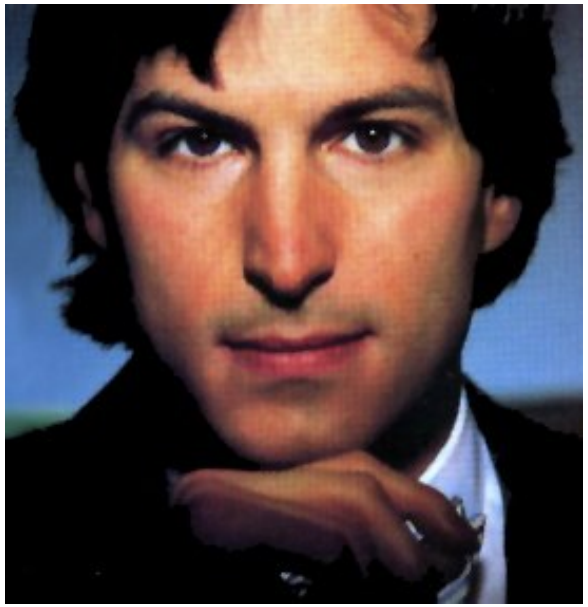
慶應義塾大学環境情報学部
田所 淳

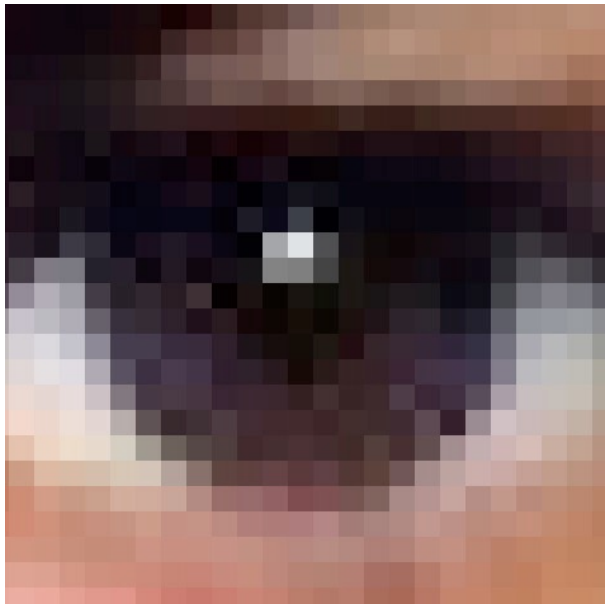
今日の内容

- 今回は、画像ファイルの読み込み、分析、再合成のやり方を解説します

ビットマップ画像のしくみ

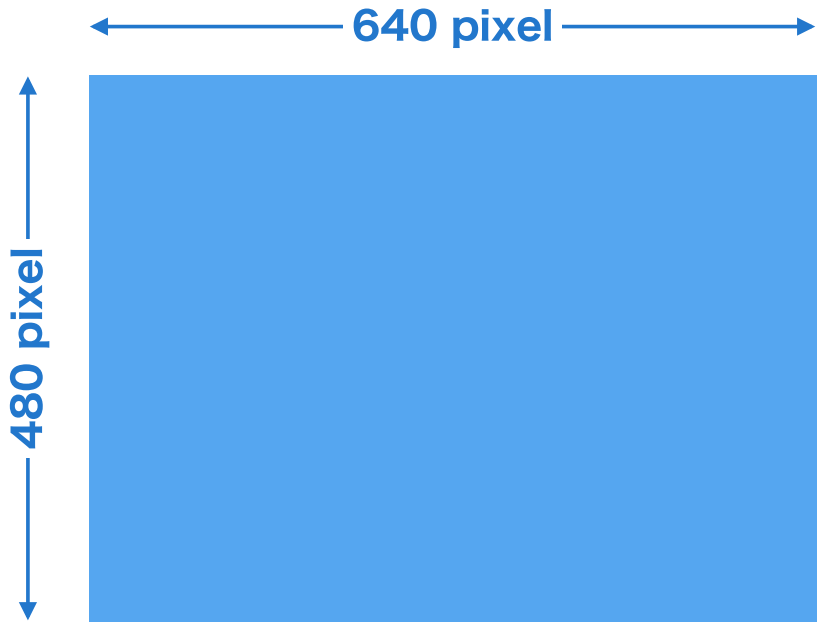
- そもそも、コンピュータモニタに、画像が映しだされる仕組み
- グリッド状に並んだ「ピクセル」という点の集合で構成される
- 巨大なエクセルの表のようなイメージ





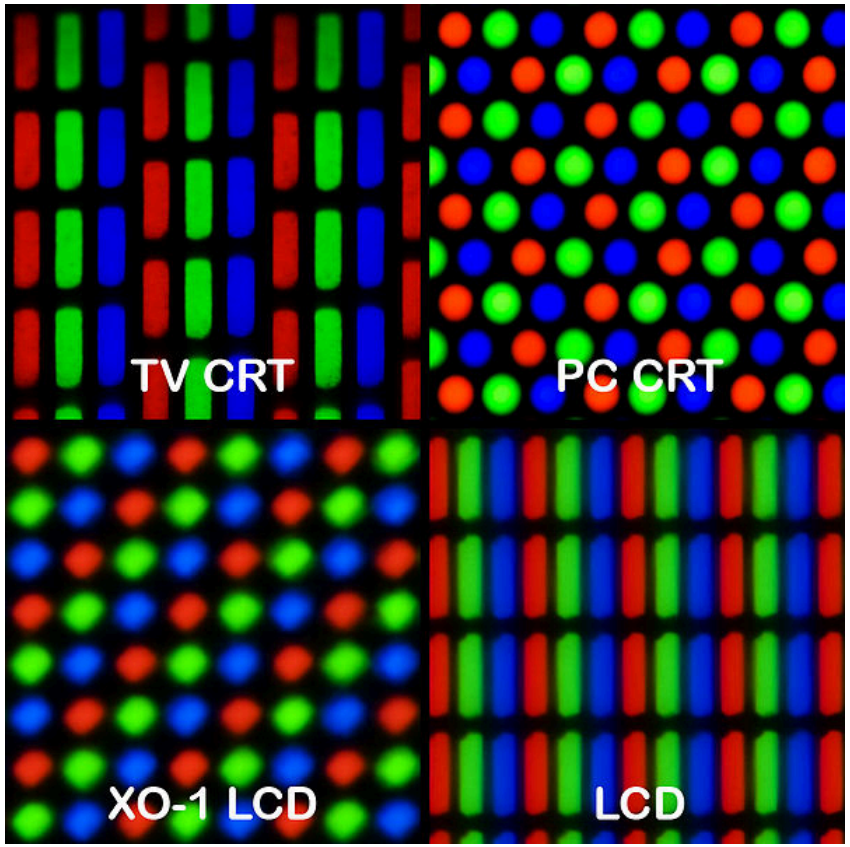
ビットマップ画像のしくみ

- 例えば、640 x 480 pixel の画像データの場合
  - 640 x 480 = 307,200
  - 307,200 個のピクセルから構成された平面



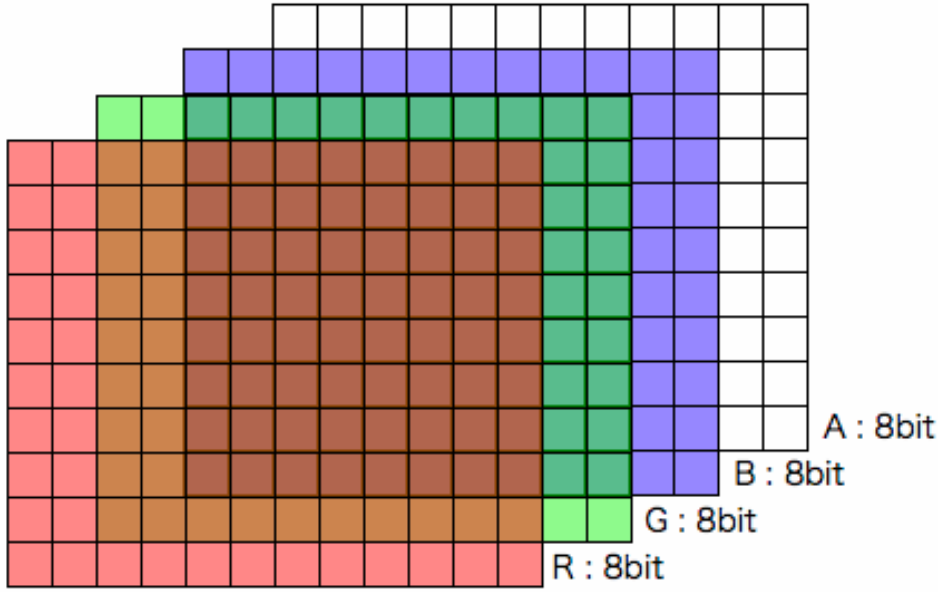
ビットマップ画像のしくみ

- 様々な種類のモニタを拡大した様子、ピクセルが見える



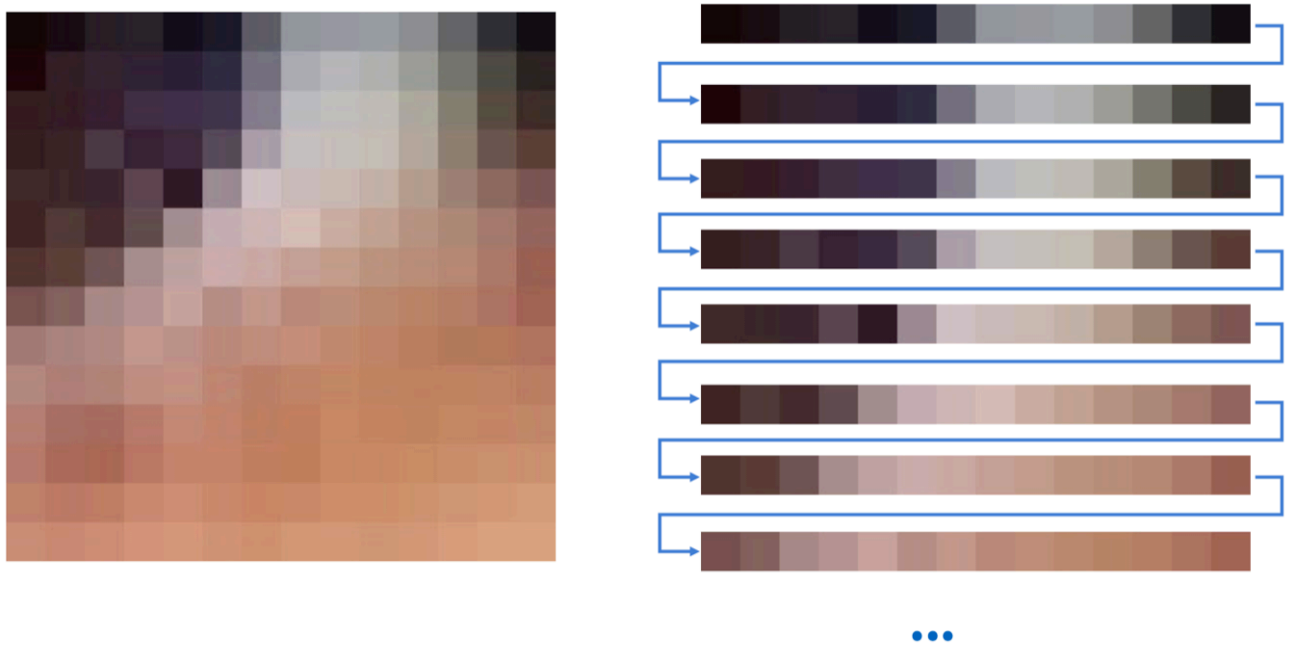
ビットマップ画像のしくみ

- それぞれのピクセルにはRGBとアルファチャンネルAのデータが格納されている(フルカラーの場合)
- それぞれのチャンネルに、8bit = 256 の階調を持つ



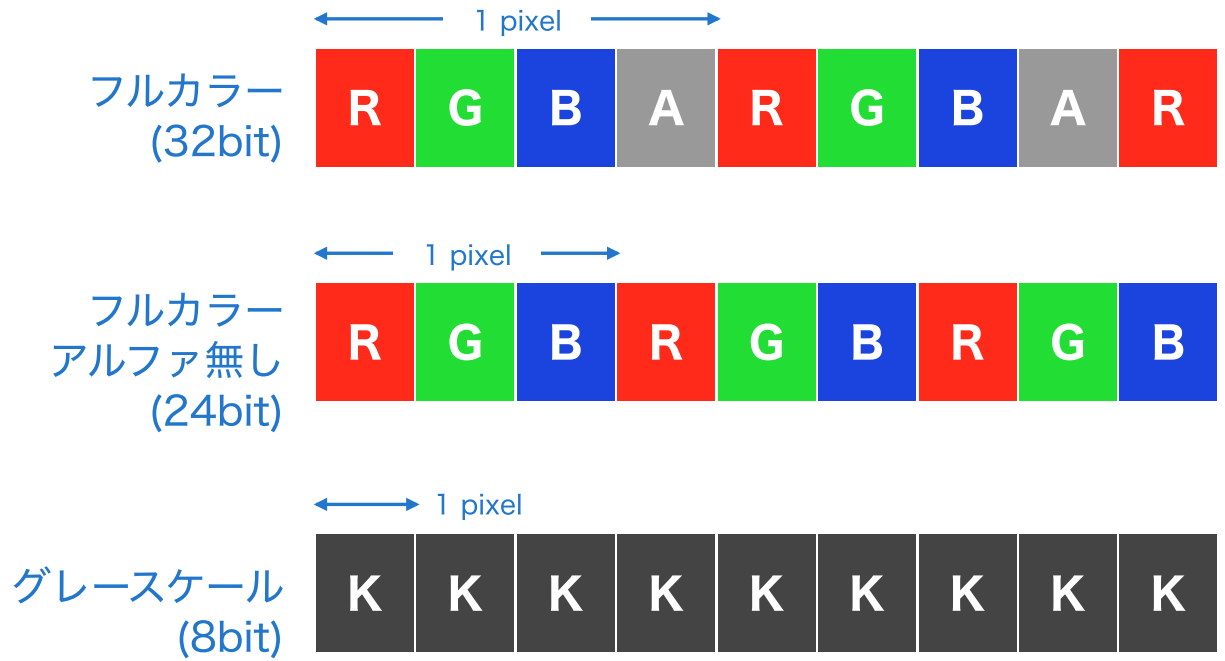
ビットマップ画像のしくみ

- 画像データのコンピュータ内での扱い
- ラスターイメージ：ビットマップ画像を点が線上に並んだもの(ラスター)の集まりとして扱う



ビットマップ画像のしくみ

- 1ピクセルの中では、画像の色数に応じてそれぞれの画素が順番に並んでいる、1画素あたり8bit (0 ～ 255) の値



画像を表示する

画像を表示する

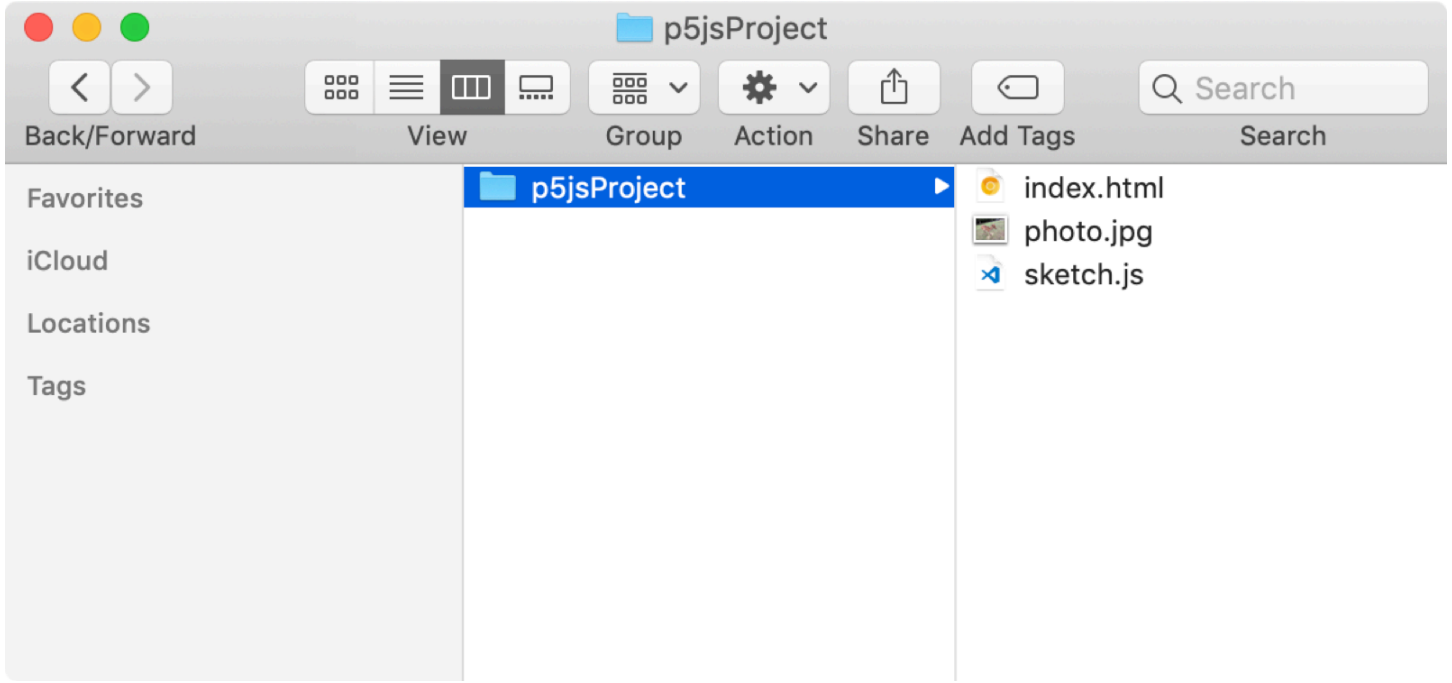
- 画像を表示するには、p5.Imageクラスを使用する
- Jpeg、GIF、PNG形式に対応
- 画像を表示するには、image() 関数を用いる
  - `image(画像ファイル名, x, y);` – 座標(x, y)を左上にして、画像を表示
  - `image(画像ファイル名, x, y, width, height);` – 座標(x, y)を左上にして、幅(width)と高さ(height)を指定して画像を表示

画像を表示する

- p5.jsで画像を表示するためには
- まず画像を配置(or アップロード)する必要あり
- 2つの方法
  - VSCodeなどのテキストエディターでローカルに使用する場合
    - 画像を適切な場所にローカルに配置
  - OpenProcessingでは
    - 画像をサーバーにアップロード

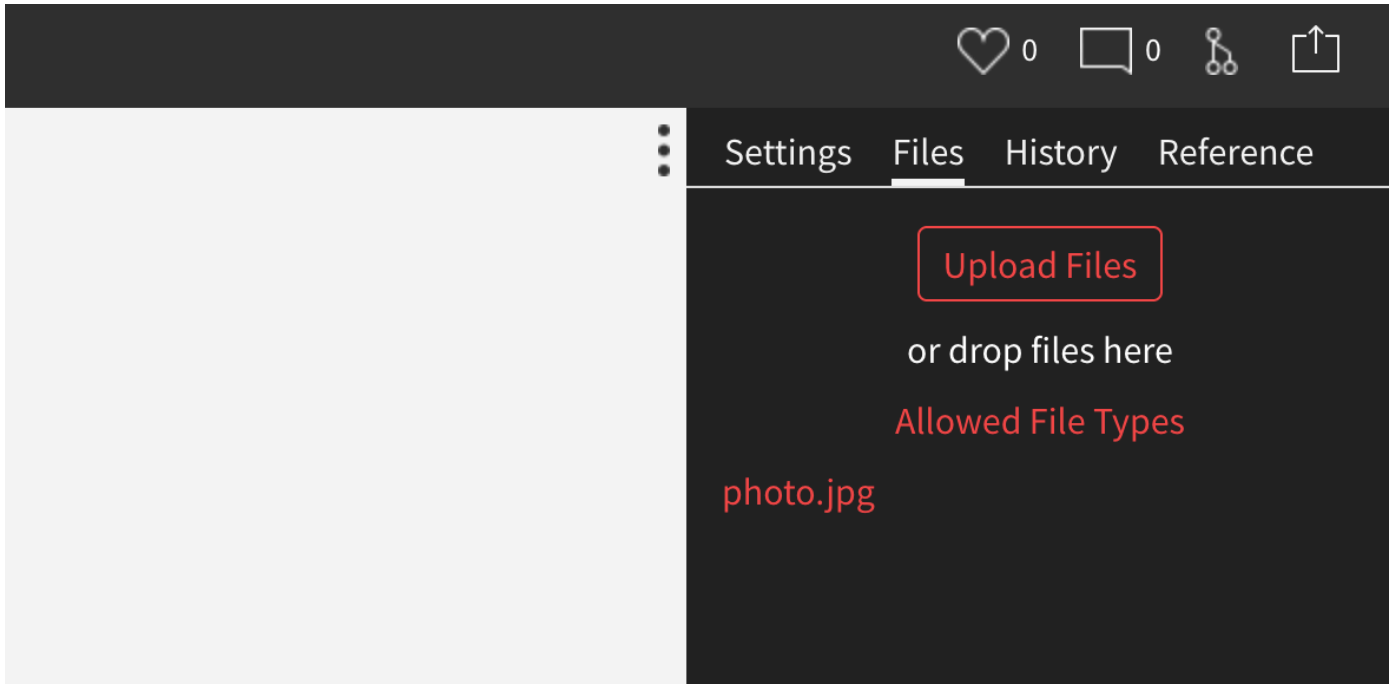
画像を表示する

- VSCodeなどで開発する場合
- 例えばシンプルに全て同じフォルダに入れる (html, js, 画像)



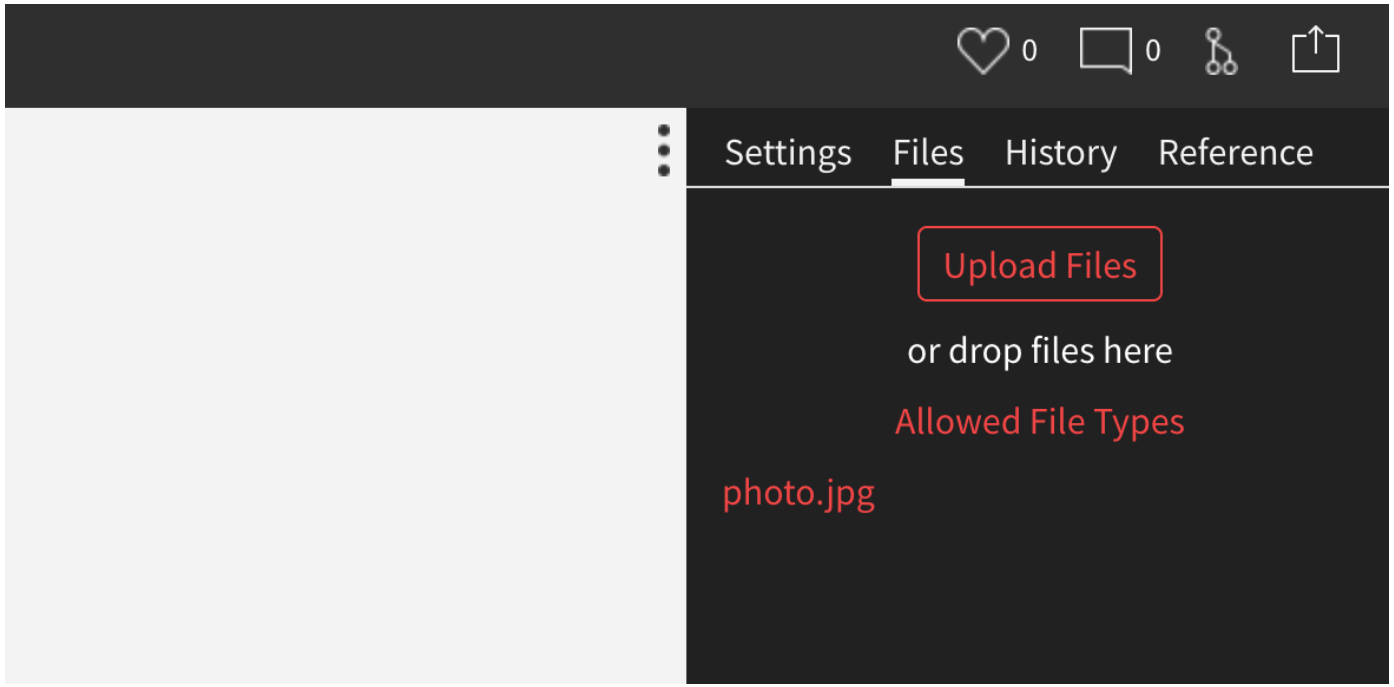
画像を表示する

- OpenProcessingの場合
- まずSketchを保存した後、コードビューで右側のメニューを表示
- Filesタブを選択



画像を表示する

- Upload Filesボタンを押して画像を選択 (またはドラッグ&ドロップ)
- 画像がアップロードされるはず



画像を読み込む

- 画像ファイルの読み込みには、loadImage() を使う
- p5.js 2.0 以降では、async と await を使って読み込む
  - setup() の前に async をつける → 処理の完了を「待つ」ことができる関数になる
  - loadImage() の前に await をつける → 画像の読み込みが終わるまで待ってから、次の行に進む
- setup() が終わってから draw() が始まるので、draw() の中でも読み込んだ画像を使える

```
async function setup() {
  createCanvas(windowWidth, windowHeight);
  img = await loadImage('./photo.jpg');
}
```

画像を表示する

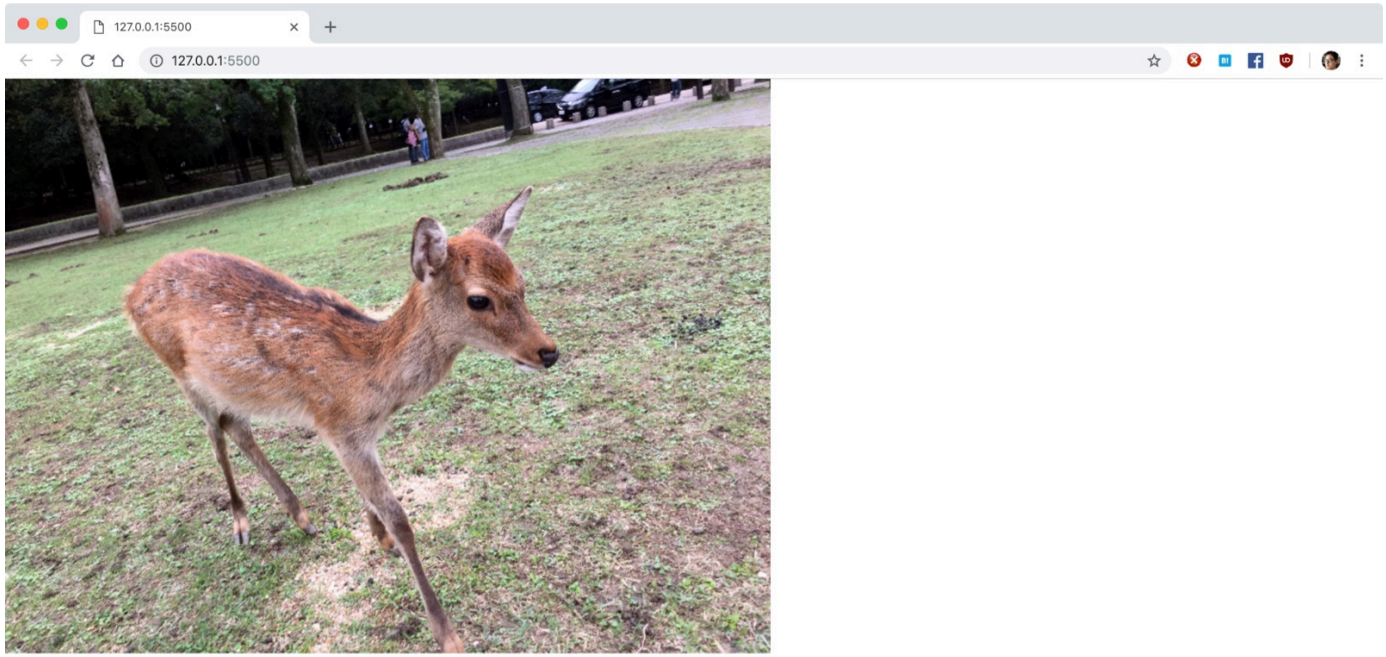
- 画像を表示するプログラム

```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //左上を起点にして画像を表示
  image(img, 0, 0);
}
```

画像を表示する

- 画像を表示 (オリジナルのサイズ)



位置とサイズを指定する

- 画像のサイズを画面の大きさにリサイズしてから表示するプログラム

```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //画像をキャンバスのサイズに合わせてリサイズ
  img.resize(width, height);
  //左上を起点にして画像を表示
  image(img, 0, 0);
}
```

画像を表示する

- 画像を表示 (拡大)



画像の色や透明度を変更する

- tint() 関数: image()で表示した画像の色や透明度を操作する
  - `tint(gray)` – 画像のレベルを変更
  - `tint(gray, alpha)` – 画像のレベルと透明度を変更
  - `tint(red, green, blue)` – RGBの値を変更
  - `tint(red, green, blue, alpha)` – RGBの値と透明度を変更

画像の色を変更する

- イメージのレベルを変更 (マウスY座標で明度が変化)

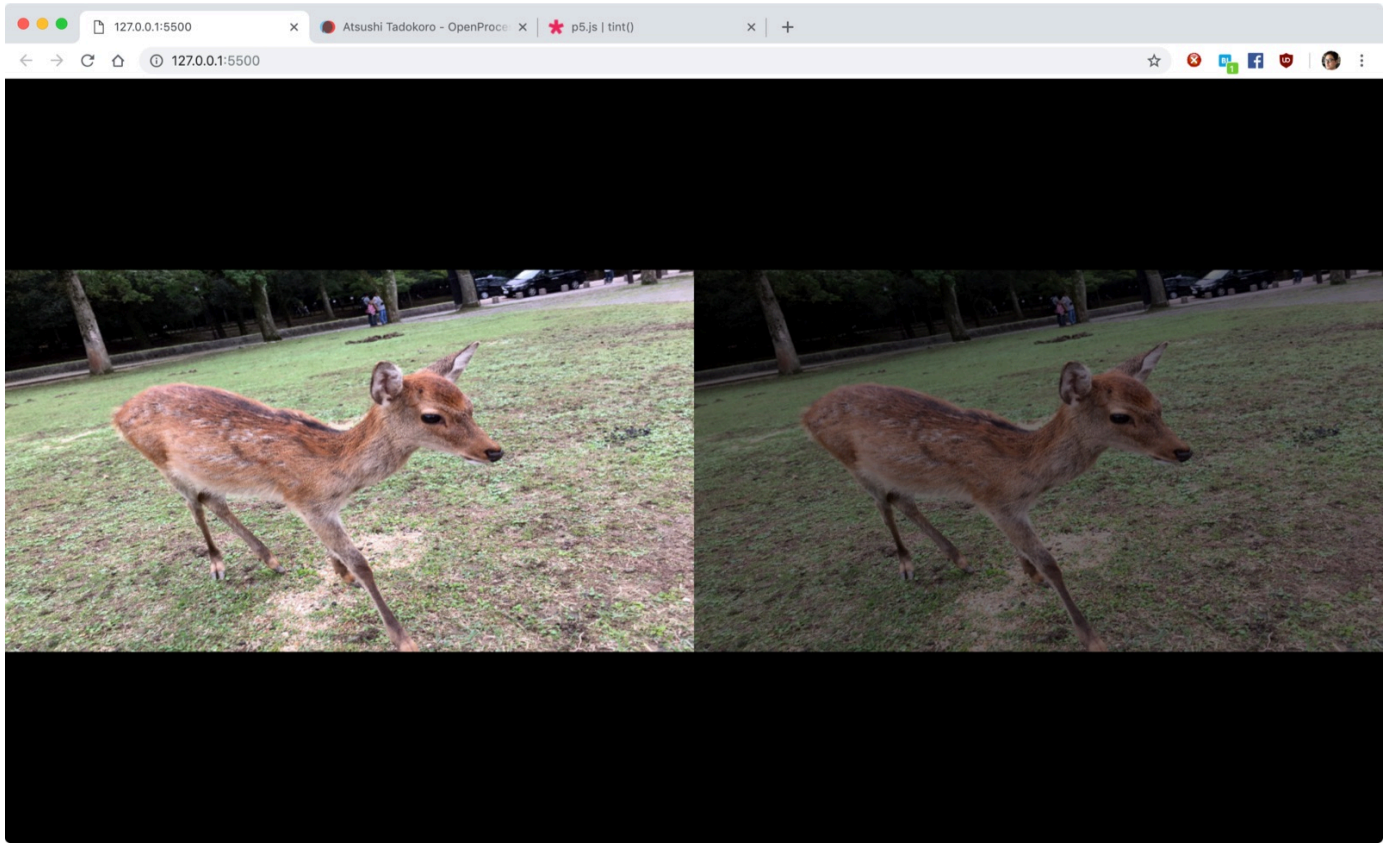
```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //画像をキャンバスのサイズに合わせてリサイズ
  img.resize(width, height);
}

function draw() {
  background(0);
  //画像の明度をマウスのY座標で変更
  tint(mouseY / float(height) * 255);
  //色を変更した画像を表示
  image(img, 0, 0);
}
```

画像の色を変更する

- イメージのレベルを変更 (マウスY座標で明度が変化)



画像の色を変更する

- イメージの色を変更 (マウスの位置で色が変化)

```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //画像をキャンバスのサイズに合わせてリサイズ
  img.resize(width, height);
}
```

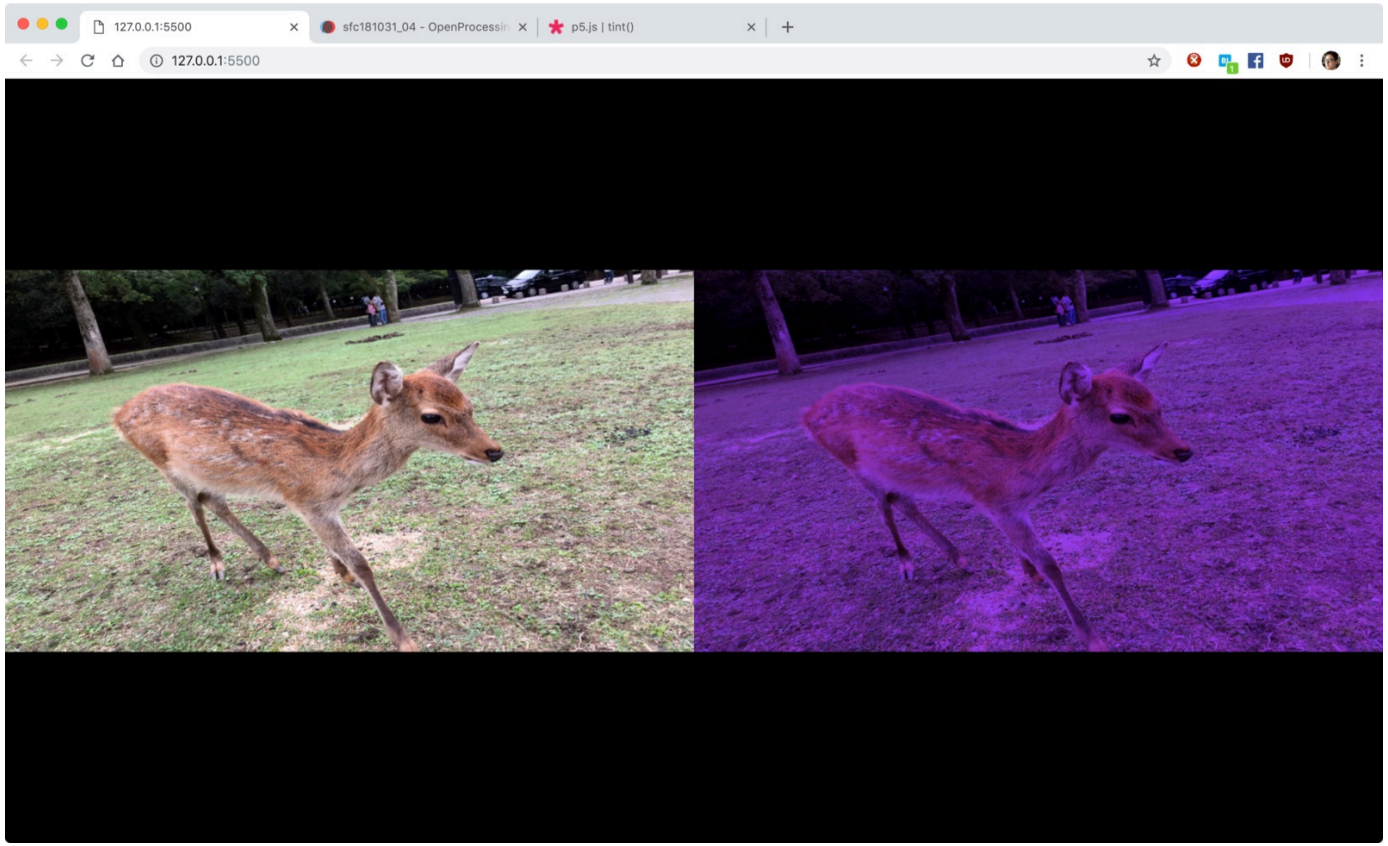
画像の色を変更する

- イメージの色を変更 (マウスの位置で色が変化)

```
function draw() {
  background(0);
  //画像のRGB値をマウスの位置で変更
  let r = mouseX / float(width) * 255;
  let g = mouseY / float(height) * 255;
  let b = 255 - mouseX / float(width) * 255;
  tint(r, g, b);
  //色を変更した画像を表示
  image(img, 0, 0, width, height);
}
```

画像の色を変更する

- イメージの色を変更 (マウスの位置で色が変化)



画像をピクセレイト

画像をピクセレイト

- いよいよ、画像のピクセルの情報にアクセスしてみる!
- `img.get(x, y)`
  - 画像imgの (x, y) の位置にあるピクセルの色の情報を取得
  - 画像全てのピクセルの色を取り出すことができる

画像をピクセレイト

- 円の大きさと色で画像を再構成する

```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //画像をキャンバスのサイズに合わせてリサイズ
  img.resize(width, height);
  //背景を黒に
  background(0);
}
```

画像をピクセレイト

- 円の大きさと色で画像を再構成する

```
function draw() {
  //色を取得する位置をランダムに決定
  let x = int(random(width));
  let y = int(random(height));
  //指定した場所の色を取得
  let col = img.get(x, y);
  fill(col, 127);
  noStroke();
  let diameter = 20;
  //円を描画
  circle(x, y, diameter);
}
```

画像をピクセレイト

- 画像の色をピックアップして同じ場所に円を描画



画像をピクセレイト

- 画像のピクセルの色情報をもとに、画像を再描画する
- ピクセレイト
- 円の大きさと色で、画像を再構成する
  - 色 - 円を描いた座標の元画像でのRGBの値
  - 直径 - 元画像の座標での色の明度
- 色の明度を求めるには、`brightness(色)`

画像をピクセレイト

- 色の明度を円の大きさに反映させる

```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //画像をリサイズ
  img.resize(width, height);
  background(0);
}
```

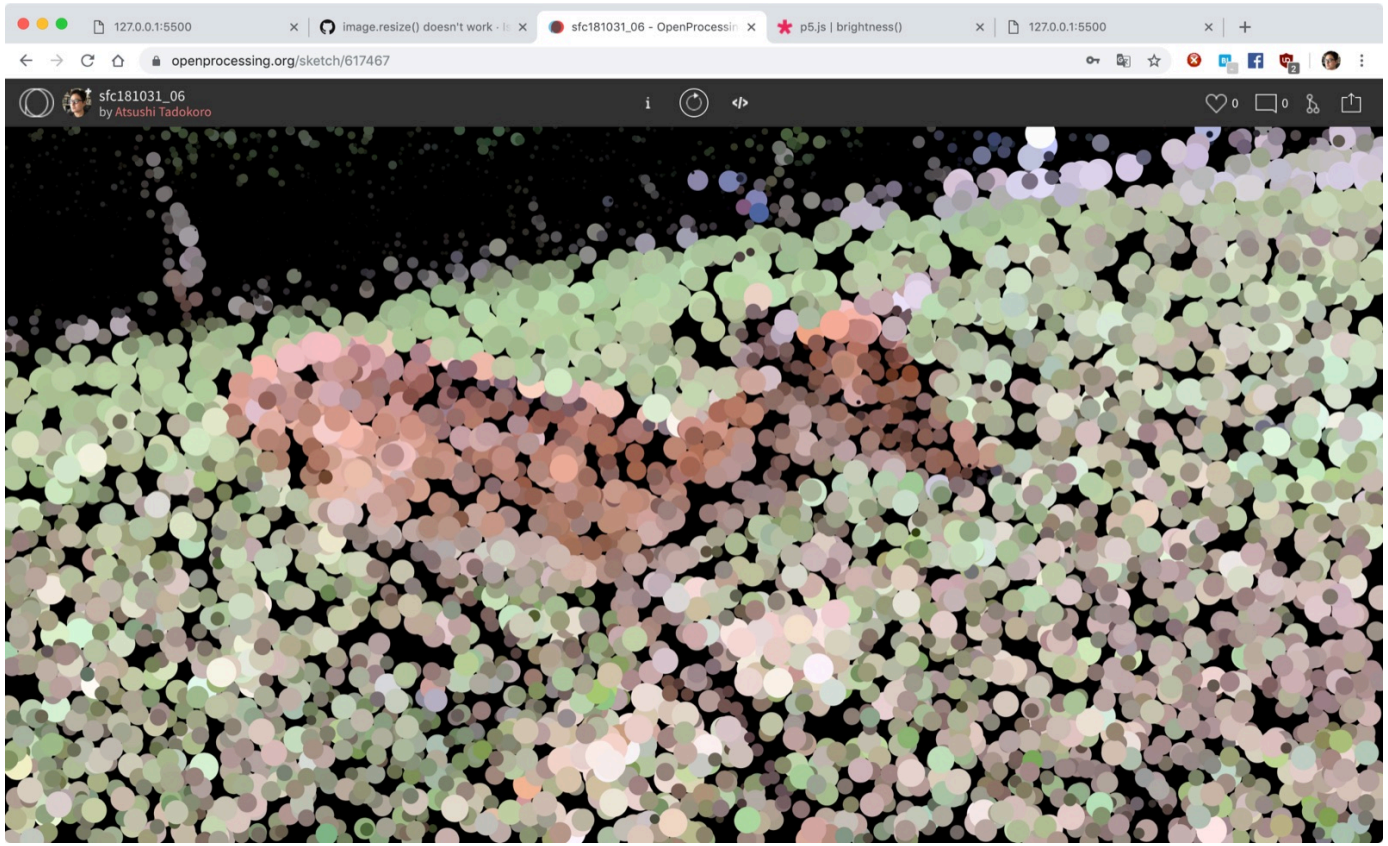
画像をピクセレイト

- 色の明度を円の大きさに反映させる

```
function draw() {
  //10倍速で描画
  for (let i = 0; i < 10; i++) {
    //色を取得する位置をランダムに決定
    let x = int(random(width));
    let y = int(random(height));
    //指定した場所の色を取得
    let col = img.get(x, y);
    fill(red(col), green(col), blue(col), 127);
    noStroke();
    //色の明度を円の直径に反映させる
    let diameter = map(brightness(col), 0, 255, 1, 80);
    //円を描画
    circle(x, y, diameter);
  }
}
```

画像をピクセレイト

- 円の大きさと色で画像を再構成する、明度がサイズに対応



画像をピクセレイト

- 色の明度ではなく、彩度を円のサイズに対応させてみる
  - 彩度が高いポイントは大きな円で描く
  - また違った効果が生まれる
  - 彩度を求めるには - `saturation(色)`
- 色に透明度を加える
  - RGBの値を個別に取り出し、そこに透明度を付加する
  - 例: `fill(red(col), green(col), blue(col), 127);`
- さらに一工夫
  - draw() で円を描く部分にfor文を用いると、描画がスピードアップする

画像をピクセレイト

- 円の大きさと色で画像を再構成する、彩度がサイズに対応

```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //画像をリサイズ
  img.resize(width, height);
  //混色を加算合成に
  blendMode(ADD);
  //背景を黒に
  background(0);
}
```

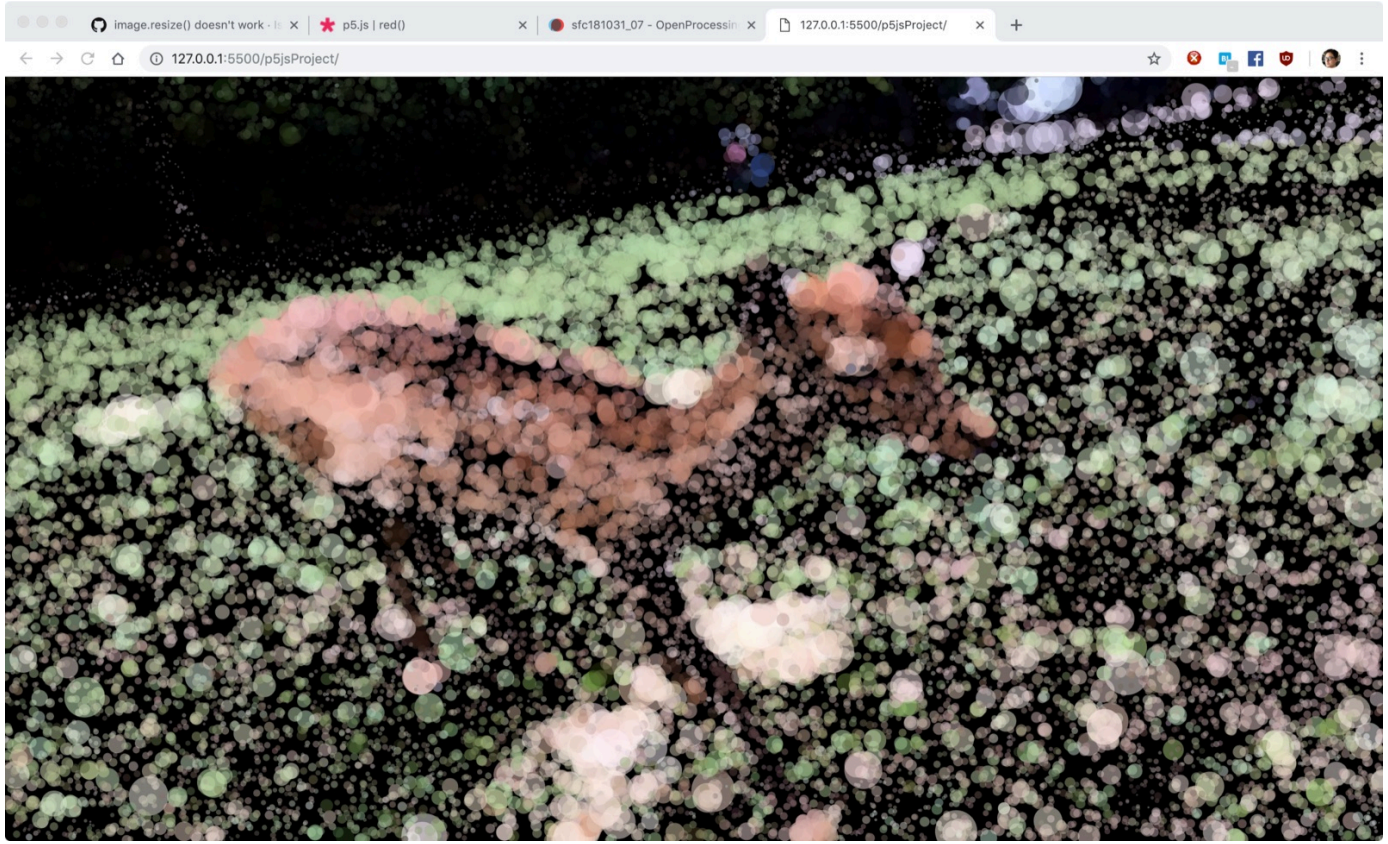
画像をピクセレイト

- 円の大きさと色で画像を再構成する、彩度がサイズに対応

```
function draw() {
  //一度に10個の円を生成
  for (let i = 0; i < 10; i++) {
    //色を取得する位置をランダムに決定
    let x = int(random(width));
    let y = int(random(height));
    //指定した場所の色を取得
    let col = img.get(x, y);
    fill(red(col), green(col), blue(col), 190);
    noStroke();
    //色の彩度を円の直径にする
    let diameter = map(saturation(col), 0, 255, 1, 40);
    //円を描画
    circle(x, y, diameter);
  }
}
```

画像をピクセレイト

- 円の大きさと色で画像を再構成する、彩度がサイズに対応



画像をピクセレイト

- 四角形の長さと回転角度で再合成してみる
  - 四角形の長さ - 明度に対応させてみる
  - 四角形の角度 - 彩度に対応させてみる

画像をピクセレイト

- 四角形の大きさと角度で、画像を再構成する

```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //画像をリサイズ
  img.resize(width, height);
  //背景を黒に
  background(0);
}
```

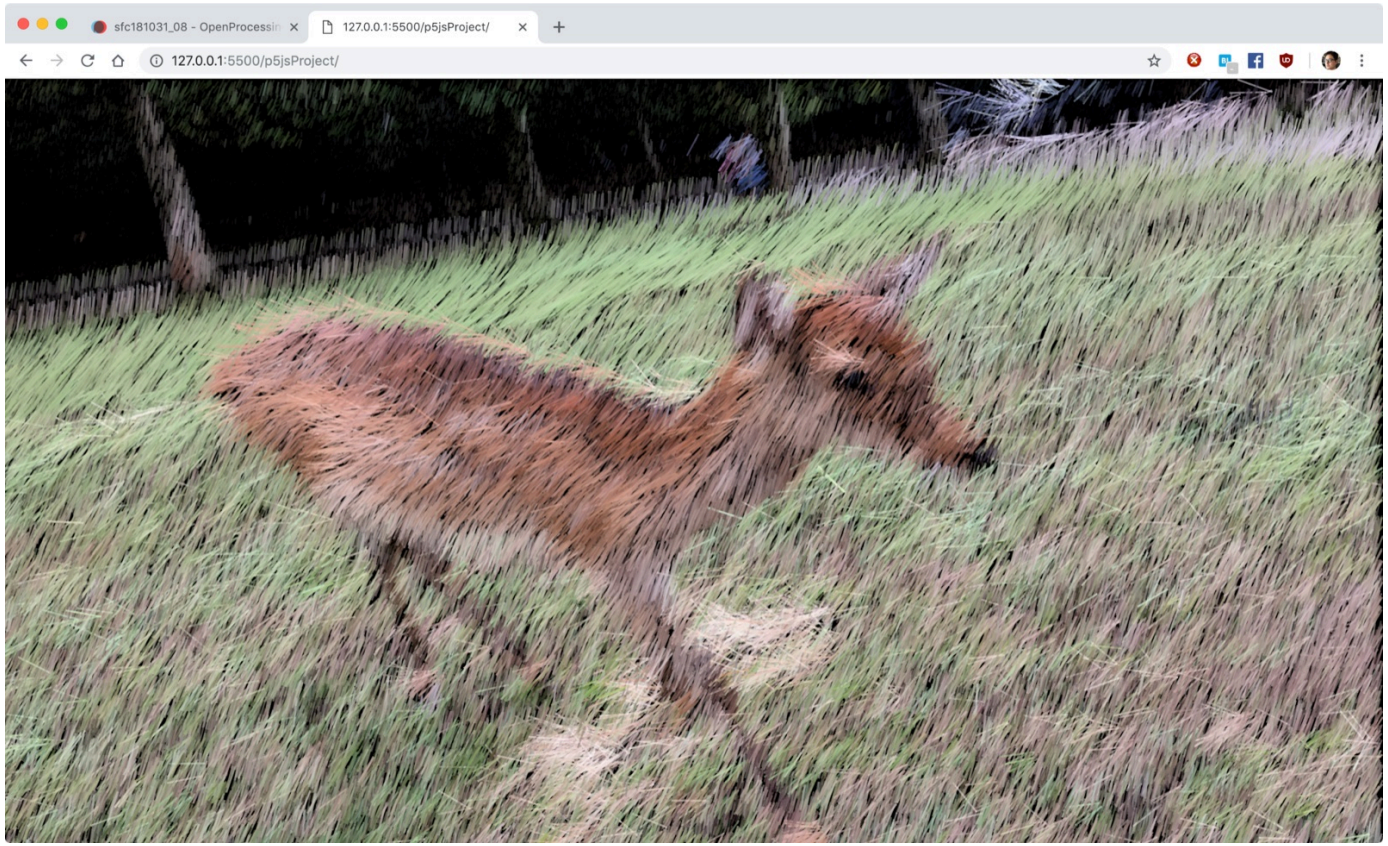
画像をピクセレイト

- 四角形の大きさと角度で、画像を再構成する

```
function draw() {
  for (let i = 0; i < 100; i++) { //100倍速
    //色を取得する位置をランダムに決定
    let x = int(random(width));
    let y = int(random(height));
    //指定した場所の色を取得
    let col = img.get(x, y);
    noStroke();
    //彩度を回転角度に反映させる
    let rotation = map(saturation(col), 0, 255, 0, 360);
    //長さは明度に反映させる
    let length = map(brightness(col), 0, 255, 0, 100);
    //色に透明度を加える
    fill(red(col), green(col), blue(col), 128);
    push();
    translate(x, y);
    rotate(radians(rotation));
    //四角形を描画
    rect(0, 0, 2, length);
    pop();
  }
}
```

画像をピクセレイト

- 四角形の大きさと角度で、画像を再構成する



応用: インタラクティブな工夫

インタラクティブな工夫

- マウスの位置によって、インタラクションしてみる
- マウスの位置の取得
  - mouseX - マウスのx座標
  - mouseY - マウスのy座標

画像をモザイク化する

- マウスの位置でモザイクの大きさが変化

```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //画像を画面の大きさにリサイズ
  img.resize(width, height);
  //背景を黒に
  background(0);
}
```

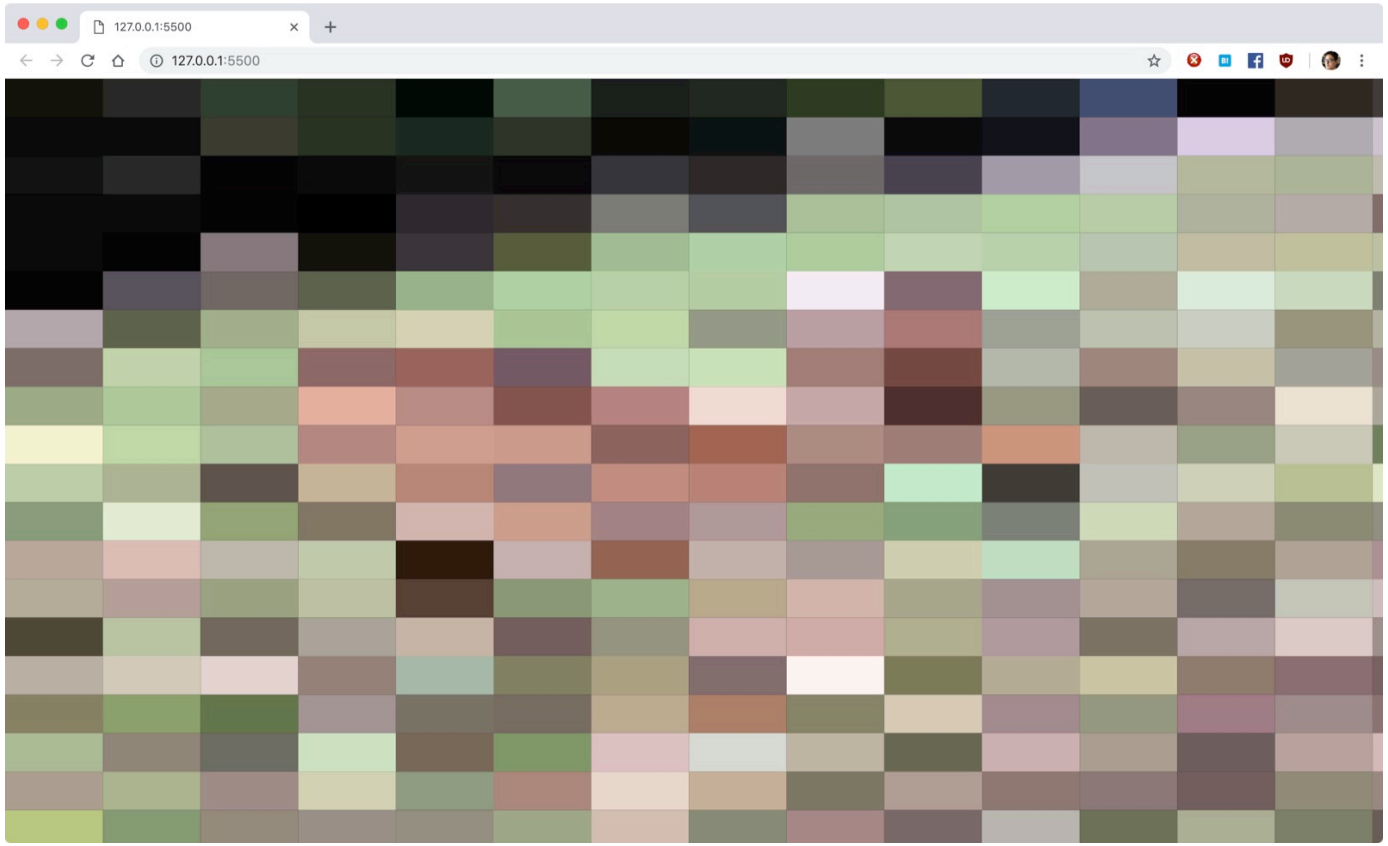
画像をモザイク化する

- マウスの位置でモザイクの大きさが変化

```
function draw() {
  background(0);
  noStroke();
  //モザイクの粒度をマウスの座標で決定
  let stepX = mouseX / 10.0;
  let stepY = mouseY / 10.0;
  //モザイクの最小値を20に
  if (stepX < 20) {
    stepX = 20;
  }
  if (stepY < 20) {
    stepY = 20;
  }
  //画面の幅と高さでループ
  for (let j = 0; j < height; j = j + stepY) {
    for (let i = 0; i < width; i = i + stepX) {
      //座標を指定して色を取得
      let c = img.get(i, j);
      //取得した色を塗りの色に
      fill(c);
      //四角形を描画
      rect(i, j, stepX, stepY);
    }
  }
}
```

画像をモザイク化する

- マウスの位置でモザイクの大きさが変化



画像ファイルをスキャンする

- img.get() を活用して、画像の特定のy座標の色だけをとりだす
- バーコード状に並べる
- 写真をスキャナーでスキャンしているイメージ

画像ファイルをスキャンする

- マウスの位置でスキャンする高さを変更

```
let img; //画像データ

async function setup() {
  createCanvas(windowWidth, windowHeight);
  //画像をロード（読み込みが終わるまで待つ）
  img = await loadImage('./photo.jpg');
  //画像を画面の大きさにリサイズ
  img.resize(width, height);
  //背景を黒に
  background(0);
}
```

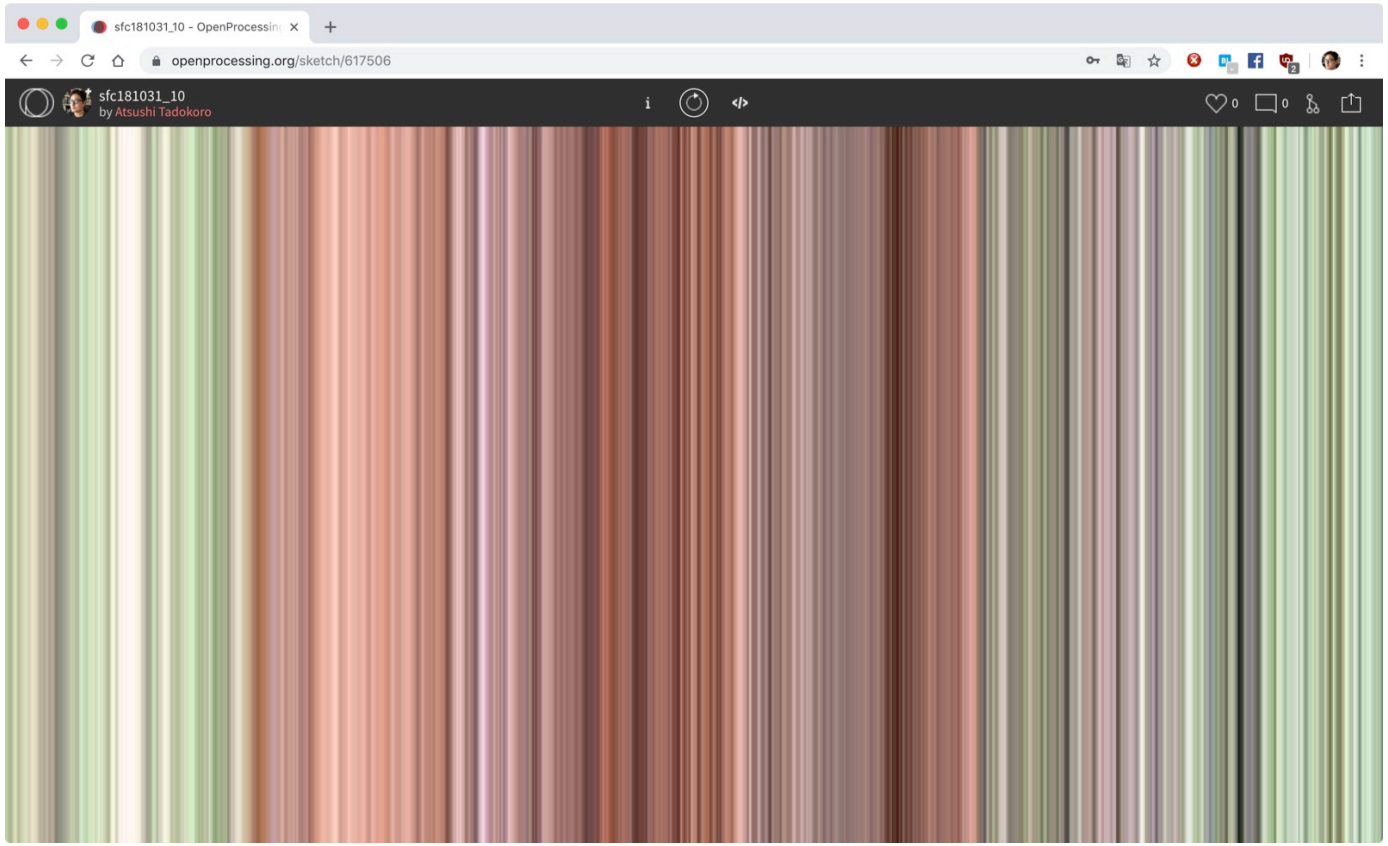
画像ファイルをスキャンする

- マウスの位置でスキャンする高さを変更

```
function draw() {
  background(0);
  noStroke();
  //画面の幅だけループ
  for (let i = 0; i < width; i++) {
    //現在のマウスのY座標の色を取得
    let c = img.get(i, mouseY);
    //取得した色を塗りの色に
    fill(c);
    //画面の高さいっぱいに幅1pixの長方形を描画
    rect(i, 0, 1, height);
  }
}
```

画像ファイルをスキャンする

- マウスの位置でスキャンする高さを変更



本日の課題

本日の課題

- テーマ: 「画像データで表現する」
- p5.jsに読み込んだ画像ファイルのデータで表現する!
  - p5.Imageに読み込んだ画像ファイルの情報から、新たなイメージを生成する
  - 読み込む画像は自由
- OpenProcessingにタイトルをつけて投稿してください!
  - 作品の(かっこいい)タイトルをつけてください!!
  - 投稿したURLをアンケートから提出
  - TAGS: sfc261030 を必ずつけること
  - 前回の提出作品で良いと思った作品を1つ選んでください